



VeloUrbe

Plataforma de Arriendo de Patinetas Eléctricas

Informe Técnico — Arquitectura de Microservicios y Código Limpio

Spring Boot 3.5 · Java 21 · Docker · PostgreSQL · JWT · Gateway → BFF → 10 Microservicios

Contexto y Solución

VeloUrbe resuelve la movilidad urbana de última milla con un ecosistema digital que permite arrendar patinetas eléctricas de forma ágil y segura. Adopta una arquitectura de **10 microservicios** tras un **API Gateway** y un **BFF** para desacoplar identidades, flota, pagos y la experiencia del cliente.

Problema

Movilidad de última milla: acceso rápido y seguro a patinetas en la ciudad.

Solución

10 microservicios de dominio tras un Gateway y un BFF, orquestados con Docker Compose.

Enfoque

Clean Code, patrón CSR, JWT stateless, Flyway y pruebas automatizadas.

Resultado

Sistema cohesivo, escalable y tolerante a fallos, listo para crecer.

Requerimientos Funcionales

RF01 Gestión de Usuarios

Registro y login emitiendo JWT firmados (HS256) con roles y permisos.

RF02 Inventario de Patinetas

CRUD completo: batería, ubicación geográfica y disponibilidad.

RF03 Gestión de Arriendos

Viajes transaccionales con marcas de tiempo y duración total.

RF04 Pagos

Ciclo completo del pago: procesar, completar, cancelar y reembolsar.

RF05 Notificaciones y Analítica

EMAIL/SMS con historial, eventos de arriendo y estadísticas de sistema.

RF06 Logística, Mantención y Soporte

GPS con búsqueda por radio, issues de mantención y tickets de soporte.

RF07 Estaciones

Capacidad, dock/undock y búsqueda por cercanía (Haversine).

RF08 Reseñas

Calificación 1–5 por arriendo (única) y promedio por patineta.

RF09 BFF Proxy + Dashboard

Proxy de /api/** a los 10 microservicios y vistas agregadas en una petición.

Requerimientos No Funcionales

Seguridad

Autenticación stateless validada por JwtAuthFilter en cada microservicio. Contraseñas con BCrypt.

Infraestructura

13 contenedores Docker (Gateway, BFF, 10 servicios y PostgreSQL). BD independiente por dominio.

Trazabilidad

Logging estructurado en JSON (logstash-logback-encoder) para indexación y monitoreo central.

Calidad de Código

Patrón CSR en los 10 servicios, DTOs inmutables (Records) y respuestas HATEOAS.

Escalabilidad

Bounded Contexts desacoplados que permiten escalar horizontalmente cada servicio.

Resiliencia

BFF con llamadas no bloqueantes vía WebClient; servicios independientes y tolerantes a fallos.

Casos de Uso Principales

1 Registro y Autenticación

El cliente envía credenciales al Gateway → pasan por el BFF al servicio de auth → se hashean y persisten → retorna un JWT que habilita el acceso a los módulos protegidos.

2 Ciclo de Vida del Arriendo

Usuario autenticado inicia un viaje → se verifica disponibilidad transaccional → el scooter pasa a "en uso" → al finalizar se libera el recurso, se contabiliza el tiempo, se procesa el pago y se puede dejar una reseña.

3 Orquestación del Dashboard

El cliente consulta al BFF → ejecuta llamadas concurrentes no bloqueantes vía WebClient a los servicios core → ensambla un payload unificado y optimizado.

Los 12 Servicios de la Plataforma

SERVICIO	PUERTO	RESPONSABILIDAD	BASE DE DATOS
api-gateway	8080	Entrada única, sin código: enruta /api/** al BFF. Aloja el Swagger centralizado.	—
bff-service	8083	Proxy a los 10 microservicios + vistas agregadas (/api/bff/**).	Sin persistencia
user-auth-service	8081	Registro, login, JWT y gestión de usuarios.	db_scooter_users
scooter-rental-service	8082	Inventario de patinetas y lógica ACID de arriendos.	db_scooter_rentals
payment-service	8084	Pagos: procesar, completar, cancelar, reembolsar.	db_scooter_payments
notification-service	8085	Notificaciones EMAIL/SMS e historial por usuario.	db_scooter_notifications
analytics-service	8086	Eventos de arriendo y estadísticas de usuario/sistema.	db_scooter_analytics
logistics-service	8087	Ubicación GPS y búsqueda de patinetas por área.	db_scooter_logistics
maintenance-service	8088	Issues de mantención: reporte y ciclo de vida.	db_scooter_maintenance
support-service	8089	Tickets de soporte: asignación y resolución.	db_scooter_support
station-service	8090	Estaciones de carga: capacidad, dock/undock, cercanía.	db_scooter_stations
review-service	8091	Reseñas 1–5 y rating promedio por patineta.	db_scooter_reviews

El BFF no cuenta como microservicio de dominio: la plataforma tiene **10 microservicios funcionales** más el Gateway y el BFF.

Diagrama C2 — Contenedores

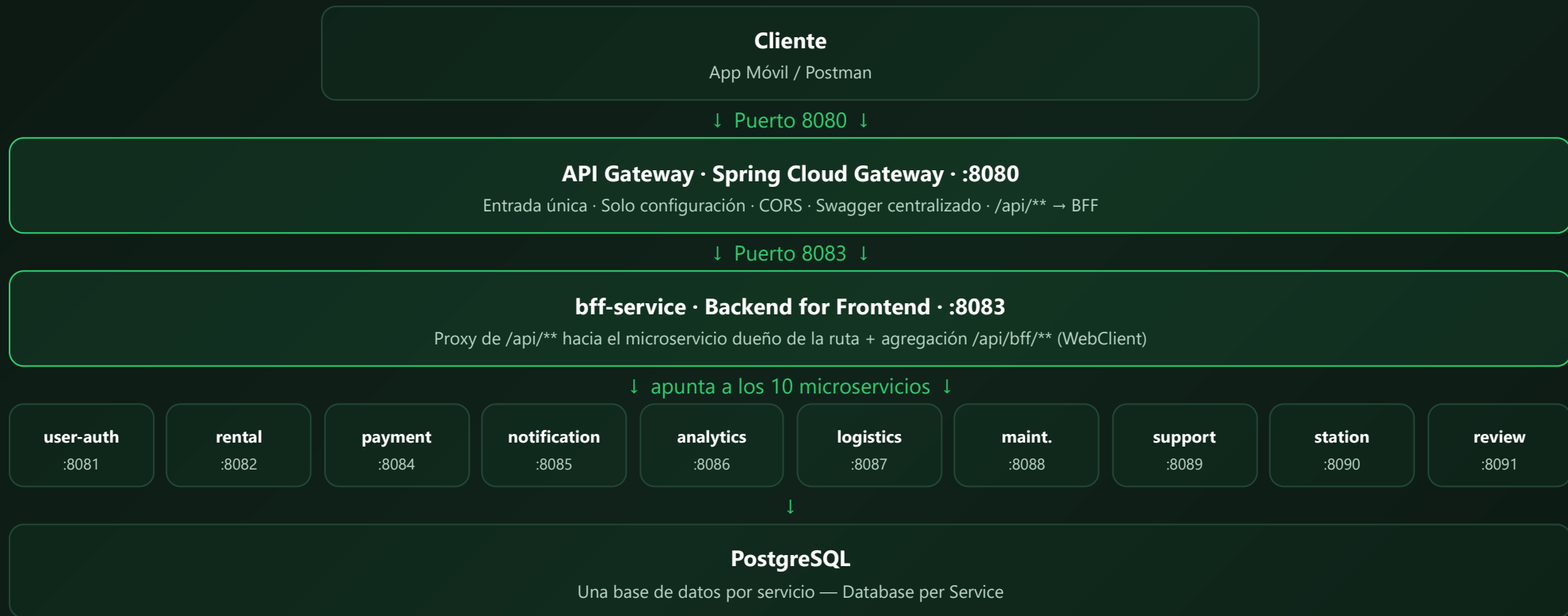


Diagrama C3 — Patrón CSR (Controller → Service → Repository)



Clean Code y Buenas Prácticas

Idioma inglés

Clases, variables, métodos y tablas íntegramente en inglés.

Javadoc

Comentarios de bloque limpios, sin `//` redundantes.

Java Records

DTOs inmutables para transferencia sin mutabilidad.

API de Streams

Transformación declarativa; sin bucles imperativos.

Patrón CSR

Controller → Service → Repository en los 10 microservicios.

Errores globales

@RestControllerAdvice + @ExceptionHandler → JSON 404/400/401.

Controllers limpios

Toda la lógica vive en la capa Service (SRP).

HATEOAS

Respuestas REST enriquecidas con enlaces de hipermedia.

Persistencia y Migraciones (Flyway)

Cada uno de los **10 microservicios** tiene su propia BD PostgreSQL (incluidas `db_scooter_stations` y `db_scooter_reviews`). Flyway versiona el esquema con DDL separado del DML.

user-auth · db_scooter_users

V1 → tabla users (DDL)
V2 → seed admin (DML)
V3 → fix password

rental · db_scooter_rentals

V1 → tabla scooters (DDL)
V2 → tabla rentals (DDL)

station / review (nuevos)

V1 → tabla stations (DDL)
V1 → tabla reviews con UNIQUE (user, rental)

Separación estricta DDL / DML

Prohibido mezclar estructura (CREATE TABLE) con datos de prueba (INSERT) en un mismo archivo. Los datos de prueba se generan con DataFaker solo bajo el perfil `dev`, garantizando despliegues limpios en producción.

Estrategia de Testing

Servicio

`@ExtendWith(MockitoExtension)`

Lógica de negocio aislada con dependencias simuladas, sin contexto Spring.

Controller

`MockMvc` standalone + Mockito

Mapeo HTTP, códigos de estado (201/200/404/401) y serialización JSON.

Contexto

`@SpringBootTest` + perfil test

El contexto levanta con H2 en memoria — la suite corre sin PostgreSQL.

user-auth-service

AuthServiceTest · UserServiceTest · AuthControllerTest · UserControllerTest
Registro, login, email duplicado, 401 inválido...

rental / payment / station / review

ScooterServiceTest · RentalServiceTest · StationServiceTest · ReviewServiceTest · PaymentControllerTest...
Arriendos, dock/undock, estación llena, rating promedio, reseña duplicada...

Cobertura total: **11 clases de test** entre servicios y controladores.

Swagger Centralizado + Postman

Swagger centralizado en el Gateway

`http://localhost:8080/swagger-ui.html` — un solo Swagger UI agrupa la documentación de los **10 microservicios + BFF** mediante un selector de definiciones. El Gateway expone cada `/v3/api-docs` bajo rutas `/docs/{servicio}` usando solo configuración (SetPath) — sin código.

Swagger por servicio

Cada microservicio mantiene su propio UI:

`localhost:8081...8091/swagger-ui.html`

Colección Postman (/postman)

Endpoints por servicio + carpetas **Stations**, **Reviews**, BFF y Vía API Gateway, con entorno de variables preconfigurado.

Flujo sugerido: Login Admin → Crear patineta → Registrar cliente → Iniciar arriendo → BFF Dashboard → Finalizar → Reseña.

Estrategia de Branching (Git)

main

Rama estable y siempre desplegable. Contiene la versión integrada de los 12 servicios.

feature/*

Una rama por funcionalidad (auth, rentals, stations, reviews...). Se integra a main mediante Pull Requests.

docs / test

Ramas de soporte para documentación, colección Postman y pruebas.

Commits semánticos

`feat:` · `test:` · `docs:` · `merge:` — historial limpio, legible y trazable. Los conflictos se resuelven con merges explícitos y documentados.

Flujo de Demostración (Docker)

Los **13 contenedores** corren localmente con Docker: **Gateway** → **BFF** → **10 microservicios** + **PostgreSQL**.

1 Registro de Usuario — 200 OK

POST /api/auth/register → crea cuenta con rol CLIENT.

2 Login — 200 OK

POST /api/auth/login → devuelve el token JWT de seguridad.

3 Sin Credenciales — 403 Forbidden

GET a endpoint protegido sin token → el sistema rechaza la petición.

4 Operación del Negocio — 201 Created

POST /api/rentals/start con token válido → arriendo creado con éxito.

El paso 3 (403 sin token) demuestra que la seguridad JWT protege el endpoint core antes de la operación válida del paso 4.

Stack Tecnológico

Java

21

Spring Boot

3.5.0

**Cloud
Gateway**

2025.0.0

**Spring
Security**

JWT

Data JPA

Hibernate

HATEOAS

links

WebFlux

WebClient

Flyway

migraciones

DataFaker

2.4.2

PostgreSQL

16

H2

tests

JWT

0.12.6

Lombok

1.18.34

SpringDoc

2.7.0

logback JSON

8.0

**JUnit /
Mockito**

tests

Gradle

9.4.1

Docker

Compose

CONCLUSIÓN

VeloUrbe — Arquitectura sólida y mantenible

La segregación en **10 microservicios**, coordinada por un **API Gateway sin código** que enruta al **BFF** (patrón Gateway → BFF → servicios), produce un sistema cohesivo, desacoplado y tolerante a fallos.

Java 21 + Records inmutables, patrón CSR verificado, Flyway, JWT stateless, Swagger centralizado y una suite de pruebas automatizadas certifican el cumplimiento de los requerimientos técnicos y de calidad exigidos.

- ✓ 10 Microservicios + BFF + Gateway
- ✓ Clean Code en inglés + Javadoc
- ✓ @RestControllerAdvice global
- ✓ Swagger centralizado + Postman
- ✓ Diagramas C2 y C3 (patrón CSR)
- ✓ Flyway DDL/DML separado
- ✓ Tests (Service / Controller / Contexto)
- ✓ Docker Compose + PostgreSQL